QUENYX VOPS HUB

CONFIDENTIAL — ARCHITECTURE

Quenyx AI Platform Bible

Quenyx vOPS HUB — v1.0.0 RC1 · Document v2.2

Document Version	2.2
Software Version	v1.0.0 RC1
Applies To	Quenyx vOPS HUB v1.0.0 RC1
Classification	Confidential — Architecture
Owner	AI Platform Engineering
Status	Released
Last Updated	2026-06-30
Document Type	Architecture reference

REVISION HISTORY

Version	Date	Notes
1.0	2026	Initial AI platform bible (through Sprint 19).
2.0	2026-06-29	RC1 alignment: Quenyx AI documented as a shared platform layer (not "QynShield AI"); explicit future-adapter list; Unified AI Workspace (Sprint 20).
2.1	2026-06-30	QynSight becomes a live AI consumer: Operations Intelligence (Sprint 21) reuses the shared runtime (provider abstraction, prompt orchestration, conversation service, audit) with no duplicated AI logic.
2.2	2026-06-30	AI Adapter Platform (Sprint 22): generalised module AI into a discoverable `AiModuleAdapter` + `AiModuleAdapterRegistry`; shared `ModuleAiNarrator` (single provider-calling point); QynAsset becomes the second production adapter (Asset Intelligence). No per-module branching.

Table of Contents

07 — Quenyx AI Platform Bible

Audience: Architects, AI engineers. **Scope:** The shared, platform-wide Quenyx AI Platform (extracted in Sprint 19, surfaced platform-wide by the Unified AI Workspace in Sprint 20), plus the QCIF AI runtime (Sprints 9–18) it consumes.

---

1. AI Platform vision

Quenyx AI is a **shared, governable AI runtime for the whole vOPS HUB — it is not "QynShield AI"** and is not a feature of any single module. Modules plug in through a thin **adapter**; the platform owns provider abstraction, skill routing, prompt orchestration, retrieval/RAG, and the capability catalog. The platform is **deterministic-first**: business logic decides *what*; AI only *renders*.

Current platform-wide capabilities: **AI Provider Abstraction**, **AI Skills**, **AI Context**, **Retrieval**, **Reasoning**, **Knowledge Graph**, **Gap Engine**, **Evidence Engine**, **Recommendation Engine**, **Copilot**, **RAG Runtime**, **Executive Platform**, provider abstraction, and **workspace isolation**.

2. Provider abstraction

All model access goes through a **provider interface** resolved by `AiProviderRegistry`. **No model is hardcoded** — names come from env (`OPENAI_MODEL` , `OPENAI_EMBEDDINGS_MODEL`). No code outside a provider class makes a direct model/HTTP call.

3. OpenAI provider 🟡

`OpenAiProvider` implements the provider interface for OpenAI (chat + embeddings). Active only when `AI_PROVIDER=openai` and `AI_ENABLED=true` ; otherwise it is never invoked.

4. Mock provider 🟢

`MockAiProvider` is the **default** (`ai.default = mock`). It returns deterministic mock responses so the entire platform runs with **zero external calls** — ideal for demos, tests, and safe defaults.

5. Provider registry

`AiProviderRegistry` maps a provider key → implementing class from `config/ai.php` . Selecting an unimplemented future provider (azure/claude/gemini/ollama/local — declared, not built) throws.

6. Skill registry

`config/ai.skills.registered` maps skill keys to classes with priorities and per-skill flags: `corpus_search` , `knowledge_graph` , `framework_mapping` , `evidence` , `gap_assessment` ,

`recommendation`. **Skills make no AI calls** — they reuse the deterministic compliance services and return AI-context payloads.

7. Skill router

The orchestrator auto-routes a request to skills by priority (higher first) and supported context. Skills are individually feature-flagged and can be disabled via env.

8. Module adapters

`AiModuleAdapterInterface` defines the contract every module implements:

```
moduleKey(): string
supportedSkills(): array
supportedContexts(): array
buildContext(AiModuleRequest $request): array
buildReasoning(AiModuleRequest $request, array $context): ReasoningOutput
buildPrompt(AiModuleRequest $request, ReasoningOutput $reasoning): AiPrompt
```

The 3-stage pipeline (**context** → **reasoning** → **prompt**) keeps deterministic work separate from model rendering.

9. QynShield adapter 🟢

`QynShieldAiAdapter` is the **first live adapter**. It wraps existing QynShield services (planner, retrieval, reasoning, prompt orchestrator, skills registry) — **no business logic was duplicated or moved**. Registered at boot with `QuenyxAiPlatform::registerAdapter(...)`.

10. QynSight — live AI consumer (Operations Intelligence, Sprint 21) 🟢

As of Sprint 21, QynSight is a **live AI consumer**. Operations Intelligence (deterministic RCA, alert explanation, incident timeline, capacity/performance/infrastructure/service-health intelligence, and the Monitoring Copilot) **reuses the shared Quenyx AI runtime** — `AiProviderRegistry`, `CompliancePromptOrchestrator`, the `AiConversation(Message)` surface, and `AiAccessAuditLogger` — via a single integration point, `OperationsAiAnalyst`. **No AI logic is duplicated**: there is no new provider registry, prompt engine, reasoning engine, or RAG engine. The Operations Intelligence services generate **deterministic, real-data evidence** and hand it to `OperationsAiAnalyst` for narration; when AI is disabled the mock provider answers (clearly flagged) while the evidence stays real. See §24. The reserved `AiModuleAdapterInterface`-style integration remains available for a future fully-routed adapter; the Sprint 21 implementation consumes the runtime directly through the Operations Intelligence services.

11. AI contract (request DTO)

`AiModuleRequest` is the generic, module-agnostic request DTO (module key, skill/context, payload). This is the single shape the platform understands, regardless of module.

12. Reasoning engine 🟢

The **Compliance Reasoning Engine** (Sprint 16) is deterministic and rule-based (`ComplianceReasoningRuleSet`). It decides what to answer **before** any AI, producing a `ReasoningOutput` (findings, applied rule IDs, explanation). Disabling it falls back to legacy skill-composed prompts. It makes **no AI or DB calls** itself.

13. Retrieval foundation 🟢/🟡

`ComplianceRetrievalService` provides deterministic retrieval over the corpus (Sprint 15). It exposes `queryDetailed()` for downstream RAG. Retrieval itself is deterministic; RAG adds optional vector recall on top.

14. RAG foundation 🟡

Sprint 17 hybrid RAG:

- `ComplianceHybridRetrievalService` merges deterministic + optional vector results, de-duplicates, and ranks; **falls back to deterministic** if a vector provider fails.
- `ComplianceRagContextBuilder` builds a **bounded, cited** context package (token budget, citations, guardrails). It makes **no AI call**.
- `OpenAiVectorRetrievalProvider` runs **metadata-only** (no fake similarity); off by default.
- **Tenant evidence is never indexed by default** (`rag.index_tenant_evidence = false`).

15. Copilot 🟢/🟡

`ComplianceCopilotService` orchestrates: scope resolve → retrieval → deterministic reasoning → (optional RAG context) → prompt orchestration → provider. **Citation-enforced** ("no source, no answer"). Runs in **mock mode** unless `AI_ENABLED=true`.

16. Demo mode 🟡

`AI_COPILOT_DEMO_MODE` adds a `demo` block exposing **existing** intelligence — reasoning trace, fired rules, citations, retrieved chunks, recommendation sources, evidence chain. It is **not chain-of-thought** and adds **no new reasoning**. Off by default.

17. Guardrails

- AI off by default; mock provider default.
- Deterministic reasoning gates the answer.
- Citations enforced; uncited chunks excluded.
- Token budget bounds context.
- No tenant-evidence embeddings by default.
- Provider-agnostic; no hardcoded models.

18. Prompt logging policy

`prompt_logging` (and Copilot's `AI_PROMPT_LOGGING_ENABLED`) default **false** — user/assistant prompt **content is never written to storage** unless explicitly enabled.

19. Conversation persistence policy

`persist_conversations` (and `AI_CONVERSATION_PERSISTENCE_ENABLED`) default **false** — Copilot can operate without storing any message content.

20. Future module adapters 🔵 / 🟣

Each HUB module can become an AI consumer by implementing `AiModuleAdapterInterface` and registering — inheriting providers, skills, prompt orchestration, retrieval/RAG, reasoning, and the capability catalog automatically. **QynShield** (compliance) and **QynSight** (Operations Intelligence, Sprint 21) are live AI consumers today. **Future AI adapters** are planned for: **QynAsset, QynRun, QynNotify, QynReact, QynKnow, QynSupport, QynBalance, and QynVA**. (`QynCore` is the platform core, not an adapter target; there is no `QynIntegrations` module.)

21. No direct DB from AI core

The AI core does not touch the database directly for business data; it consumes **deterministic service outputs** (via skills/adapters). DB access lives in the compliance/services layer, not in the AI orchestration core.

22. No direct provider calls outside providers

Reaffirmed and **verified by static scan**: only provider classes (and the metadata-only vector provider, via the registry) reference the model SDK. (The legacy QynSight knowledge-base agent `Services/OpenAI/OpenAIService` predates the platform and is a separate, isolated service.)

23. Capability catalog

`GET /api/ai/platform/capabilities` (`QuenyxAiCapabilityCatalog`) dynamically returns registered **modules** (adapters), **skills**, **providers**, **reasoning/retrieval/RAG** status, supported contexts, and the HUB-wide `module_catalog` (production/reserved/planned), independent of UI visibility.

Quenyx AI — Unified AI Workspace (Sprint 20)

> **RC1.1:** surfaced in the UI as **Quenyx AI** (the enterprise AI control center). Internal/codename remains *Unified AI Workspace*; API (`/api/ai/*`) and SPA (`/ai-workspace/*`) paths are unchanged and a branded `/quenyx-ai/*` alias redirects to them.

A platform-level surface that exposes the shared AI runtime to every workspace through flat `/api/ai/*` endpoints (see API Reference §17). It **reuses** the existing runtime rather than adding new AI logic:

- **Conversations / Chat / History** reuse `AiConversation(Message)` + `AiConversationRepository`; message execution goes through `AiProviderRegistry` + `CompliancePromptOrchestrator` (mock provider until `ai.feature_flags.enabled`). Conversation **metadata + token counts** are always persisted for this surface; message **content** only when `prompt_logging` is enabled.
- **Skills / Capabilities** read `QuenyxAiPlatform` / `QuenyxAiCapabilityCatalog` (Sprint 19) — no duplicated catalog.
- **Usage / Costs** are **derived** from real token counts; costs multiply by an optional `config('ai.workspace.pricing')` table and never fabricate currency.
- **Prompt Templates** (`ai_prompt_templates`), **Provider Settings** (`ai_provider_settings` , encrypted, write-only secrets), and **Permissions** (`ai_workspace_permissions`) add the missing governance concepts only.
- **Provider catalog & governance (RC1.1)**: `App\Services\Ai\AiProviderCatalog` declares the customer-visible provider catalog (OpenAI, Anthropic, Gemini, Azure OpenAI, OpenRouter, Mistral, Cohere, xAI Grok, Ollama, LM Studio, vLLM, LiteLLM, Hugging Face, Custom OpenAI-compatible). The catalog is **metadata only** — a provider is `executable` only when a real adapter exists in `config('ai.providers')` (today: **OpenAI**; plus the dev-only **mock**), and `platform_configured` only when real credentials are present. `AiProviderRegistry::defaultKey()` never returns `mock` in production: it prefers an explicit `AI_PROVIDER` , then OpenAI when configured, then `mock` only in local/testing, otherwise `''` (an honest "no provider configured" state). A real **test-connection** probe (`POST /api/ai/providers/{uuid}/test`) runs the adapter's `health()` for executable providers and reports `not_executable` for the rest — connectivity is never fabricated. Mock is excluded from the provider list outside local/testing.
- **Audit**: conversations, provider updates, template changes, and permission changes are logged via `AiAccessAuditLogger` / `AiWorkspaceAuditLogger` (never secrets or content).
- **Memory**: no durable AI memory store exists yet, so the Memory surface honestly reports "not enabled" instead of fabricating data.

Master switch: `ai.feature_flags.workspace_enabled` (env `AI_WORKSPACE_ENABLED`, default on; safe because chat falls back to the mock provider and metrics read 0 with no activity).

24. QynSight Operations Intelligence (Sprint 21)

Operations Intelligence transforms QynSight from a monitoring platform into an **operations intelligence platform**: the AI *understands and explains* operational data instead of merely answering free-form questions. It is an **additive intelligence layer** over the frozen monitoring engine and **reuses the shared AI Platform** end-to-end.

Reuse, not duplication. A single service, `App\Services\Observe\Intelligence\OperationsAiAnalyst`, is the *only* integration point with the AI runtime. It uses the existing `AiProviderRegistry` (same provider abstraction), `CompliancePromptOrchestrator` (same grounded-prompt assembly with citations/guardrails), the Sprint 20 `AiConversation` / `AiConversationRepository` (so every Copilot thread is a real Quenyx AI conversation), and `AiAccessAuditLogger` (same audit). No new provider registry, prompt engine, reasoning engine, or RAG engine was created.

Deterministic evidence first. Every narrative is grounded in real, deterministic evidence:

- `OperationsEvidenceCollector` — reads current hosts, services, alerts, capacity, metrics, topology.
- `RootCauseService` — deterministic layered scoring (CPU → memory → storage → database → application) over real signals; never invents causal chains.
- `IncidentTimelineService` — builds timelines from **actual event timestamps**.
- `CapacityAdvisorService` — reuses Capacity Planning + per-host forecasting from historical metrics.
- `InfrastructureImpactService` — dependencies, SPOF, and blast radius from existing topology.
- `PerformanceAdvisorService` — hotspots, trends, anomalies, slow services from real metric history.
- `AlertExplanationService` / `OperationsIntelligenceService` — compose evidence, then narrate.

UUID-only & deterministic ids. QynSight stores numeric ids internally; the AI surface is UUID-only. `OperationsEntityId` derives a deterministic **UUIDv5** per entity (type + workspace + id) and `OperationsEntityResolver` maps it back within a workspace — **no schema change, no numeric ids exposed**.

Governance. Every request is workspace-scoped, gated by the `qynsight` entitlement, monitoring RBAC, and the `can_use_ai` capability, and is audited, provider-logged, conversation-logged, and rate limited (`throttle:ai-workspace`). Endpoints: see API Reference §18.

25. AI Adapter Platform (Sprint 22)

Sprint 22 generalised module AI into a **reusable adapter framework** so every future module (QynRun, QynNotify, QynReact, QynKnow, QynSupport, QynBalance, QynVA) plugs in the same way, with **zero per-module branching** in the platform. The flow is:

```
Quenyx AI → AI Adapter Registry → {QynSight, QynAsset, …} → Capabilities → Actions → Shared AI P
```

The contract — `App\Contracts\QuenyxAI\AiModuleAdapter`. A coarse, discovery-oriented contract (companion to the fine-grained `AiModuleAdapterInterface` 3-stage pipeline). It exposes module metadata (`moduleKey/Name/Description/Category/Version/Icon`), `capabilities()` , `supportedEntities()` , `supportedSkills()` , `supportedProviders()` , `availableActions()` , and `buildContext()` . The metadata methods were added **backward-compatibly** via `AbstractAiModuleAdapter` , which supplies defaults so existing adapters keep working and new adapters override only what they need (Sprint 21's `QynSightAiAdapter` was upgraded with metadata and **no behavior change**).

The registry — `App\Services\QuenyxAI\AiModuleAdapterRegistry`. A process-wide singleton that discovers, registers, and resolves adapters, and aggregates their metadata, capabilities, actions, and entities. Modules register one line each in `AppServiceProvider::boot()` . It holds **no business logic** and **calls no provider** — it is pure discovery.

Single provider-calling point — `App\Services\Ai\ModuleAiNarrator`. The shared narration service that every module uses to narrate deterministic evidence. It reuses `AiProviderRegistry` , `CompliancePromptOrchestrator` (grounding guardrails + citations), and `AiAccessAuditLogger` . There is now **exactly one** place in the codebase that talks to a provider for module intelligence; Sprint 21's `OperationsAiAnalyst` was refactored to **delegate** to it (behavior preserved).

Discovery API (entitlement-filtered, RBAC-gated, audited). `GET /api/ai/adapters` , `GET /api/ai/adapters/{module}` , `GET /api/ai/adapters/capabilities` , `GET /api/ai/actions` — every response is scoped by a required `workspace` UUID and filtered to the adapters the workspace is entitled to (no data leakage). The AI Workspace builds navigation/actions from these, never from a hard-coded module list.

Adding a module = two steps: implement an `AiModuleAdapter` (reusing the module's domain services and `ModuleAiNarrator`), then register it. Nothing else in the platform changes. See the **AI Adapter Developer Guide (doc 23)**.

Sprint 23 — QynRun & QynReact adapters + cross-module orchestration

Two new production adapters register the same way (no platform change): **`qynrun`** (Enterprise Automation) and **`qynreact`** (Incident Intelligence) — bringing the registry to four production adapters (QynSight, QynAsset, QynRun, QynReact). Both narrate exclusively through the shared `ModuleAiNarrator` ; no AI/provider/orchestration logic is duplicated.

Cross-module intelligence (no branching). QynReact's `CrossModuleOrchestrator` realizes the flow *Alert → Asset → Incident → Automation → Knowledge → Resolution* by iterating the **AI Adapter Registry** and asking each entitled module to `buildContext()` . It excludes the calling module (`qynreact`) to

avoid recursion. A future module joins incident reasoning automatically once it registers an adapter — there is still no `if (module == …)` anywhere.

Automation Learning as AI evidence. QynRun/QynReact recommendations cite the auditable `AutomationLearningService` aggregates (success/failure/rollback rates, typical duration). There is **no model training and no hidden state** — the "learning" is inspectable, workspace-scoped history. See the **Automation Platform Guide (doc 24)**, **QynRun Guide (doc 25)**, **QynReact Guide (doc 26)**.

Sprint 24 — Knowledge, Service Desk & Notification adapters

Three new production adapters register the same way (no platform change): **`qynknow`** (Enterprise Knowledge), **`qynsupport`** (Service Desk / Ticket Intelligence), and **`qynnotify`** (Notification Intelligence) — bringing the registry to seven production adapters (QynSight, QynAsset, QynRun, QynReact, QynKnow, QynSupport, QynNotify). All narrate exclusively through the shared `ModuleAiNarrator`; no AI/provider/orchestration logic is duplicated, and `ai_candidate` is enabled for the three modules in `config/quenyx_ai.php`.

Knowledge Assistant. `QynKnowIntelligenceService` supports Explain, Summarize (KB/incident/executive/technical), Find related, and Generate (KB/runbook) drafts. Every answer is grounded in real evidence retrieved via `EnterpriseSearchService` and cross-module context from the `CrossModuleOrchestrator`; drafts are **editable and never fabricated**, never auto-applied.

Ticket & Notification Intelligence (evidence-based). Suggestions (category/priority/impact/assignee/SLA, digests, executive summaries) are computed deterministically from real rows and only then narrated; when there is no history the services say **"insufficient evidence"** rather than inventing facts. With AI disabled the mock provider answers (flagged) while the evidence stays real.

Registry-driven knowledge. The `KnowledgeSourceRegistry` mirrors the AI Adapter Registry pattern: sources self-describe and self-register, so adding a provider never touches search/graph/timeline code. See the **Enterprise Knowledge Guide (doc 28)**, **Service Desk Guide (doc 29)**, **Notification Guide (doc 30)**, **Collaboration Guide (doc 31)**, **Global Timeline Guide (doc 32)**.